U24AI047 AI LAB ASSIGNMENT-4

DATE: 29/01/2026

Q1 Use the city map given in Q1 of assignment 1 and implement the best first search algorithm to reach your destination and compare your results in terms of the number of possible paths explored.

```
class Node:
    def __init__(self, state, parent=None, action=None, path_cost=0):
        self.state = state
        self.parent = parent
        self.action = action
        self.path_cost = path_cost

class PriorityQueue:
    def __init__(self, f):
        self.data = []
        self.f = f

    def add(self, node):
```

```
        self.data.append(node)
        self.data.sort(key=self.f,reverse=True) # Sort by f(node) = path_cost

    def pop(self):
        return self.data.pop(0)

    def top(self):
        return self.data[0]

    def is_empty(self):
        return len(self.data) == 0

# City mapping
cities = {
    0: "Chicago",
    1: "Detroit",
    2: "Cleveland",
    3: "Indianapolis",
    4: "Columbus",
    5: "Pittsburgh",
    6: "Buffalo",
    7: "Syracuse",
    8: "New York",
    9: "Philadelphia",
    10: "Baltimore",
    11: "Boston",
    12: "Providence",
    13: "Portland"
}

n = 14

# basically build empty adjacency matrix
#this list comprehension will make 14 X14 matrix for me and initialize all
the  values as 0
adj = [[0 for _ in range(n)] for _ in range(n)]


#how have i defined this?
# index_node_1 -> index_node_2, the direct travelling cost
edges_clean = [
    (0,1,283), (0,2,354), (0,3,182),     # Chi: Det 283, Cle 354, Ind 182
```

```
    (1,6,256), (1,2,169),              # Det: Buf 256, Cle 169
    (2,6,189), (2,5,134), (2,4,144),     # Cle: Buf 189, Pit 134, Col 144 (Chi 345
mismatch? Q1.py says Cle-Chi 345, Chi-Cle 354? Asymmetric? Map is usually
symmetric. I'll use 354 as in snippet or check LAB-1 dict 354)
    (3,4,176),                         # Ind: Col 176
    (4,5,185),                         # Col: Pit 185
    (5,6,215), (5,9,305), (5,10,247),    # Pit: Buf 215, Phi 305, Bal 247
    (6,7,150),                         # Buf: Syr 150
    (7,8,254), (7,11,312),              # Syr: NY 254, Bos 312
    (8,9,97), (8,12,181), (8,11,215),    # NY: Phi 97, Pro 181, Bos 215
    (9,10,101),                        # Phi: Bal 101
    (11,12,50), (11,13,107)             # Bos: Pro 50, Por 107
]

# Build adjacency matrix from edges_clean
for u, v, w in edges_clean:
    adj[u][v] = w
    adj[v][u] = w   # undirected graph


class Problem:
    def __init__(self, initial, goal):
        self.INITIAL = initial # Start City Index
        self.goal = goal # Goal City Index

    def is_goal(self, state):
        return state == self.goal

    def ACTIONS(self, state):
        actions = []
        for i in range(n):
            if adj[state][i] != 0:
                actions.append(i)
        return actions  # List of the directly next city indices

    def RESULT(self, state, action):
        return action # In this graph, action is the next state index

    def ACTION_COST(self, s, action, s_prime):
        return adj[s][s_prime]

def EXPAND(problem, node):
```

```
    s = node.state
    for action in problem.ACTIONS(s):
        s_prime = problem.RESULT(s, action)
        cost = node.path_cost + problem.ACTION_COST(s, action, s_prime)
        yield Node(state=s_prime, parent=node, action=action, path_cost=cost)

def BEST_FIRST_SEARCH(problem, f):
    node = Node(problem.INITIAL) # Start Node

    frontier = PriorityQueue(f)
    frontier.add(node)

    # reach dict stores state -> node
    reached = [None] * n
    reached[problem.INITIAL] = node
    explored = 0

    while not frontier.is_empty():
        node = frontier.pop()
        explored += 1

        if problem.is_goal(node.state):
            return node, explored, reached

        for child in EXPAND(problem, node):
            s = child.state
            if reached[s] is None or child.path_cost < reached[s].path_cost:
                reached[s] = child
                frontier.add(child)

    return None, explored, reached

# Path Cost (UCS)
# Using path cost as per the snippet's logic f(node) => node.path_cost (Uniform
Cost Search)
def f(node):
    return node.path_cost

def get_path(node):
    path = []
    while node:
```

```
        path.append(cities[node.state])
        node = node.parent
    return path[::-1]


if __name__ == "__main__":
    #our main code
    start = 7   # Syracuse
    goal = 0    # Chicago

    problem = Problem(start, goal)
    solution, explored_count, reached_table = BEST_FIRST_SEARCH(problem,
f)

    if solution:
        print("Uniform Cost Search Path:")
        print(" -> ".join(get_path(solution)))
        print("Total cost:", solution.path_cost)
        print("Number of paths explored:", explored_count)

        print("\nLookup Table (Reached States):")
        print(f"{'City Index':<12} | {'City Name':<15} | {'Cost':<6} | {'Parent'}")
        print("-" * 55)
        for i in range(len(reached_table)):
            node = reached_table[i]
            city_name = cities[i]
            if node:
                parent_name = cities[node.parent.state] if node.parent else "None"
                print(f"{i:<12} | {city_name:<15} | {node.path_cost:<6} |
{parent_name}")
            else:
                print(f"{i:<12} | {city_name:<15} | {'None':<6} | -")
    else:
        print("No path found.")
```

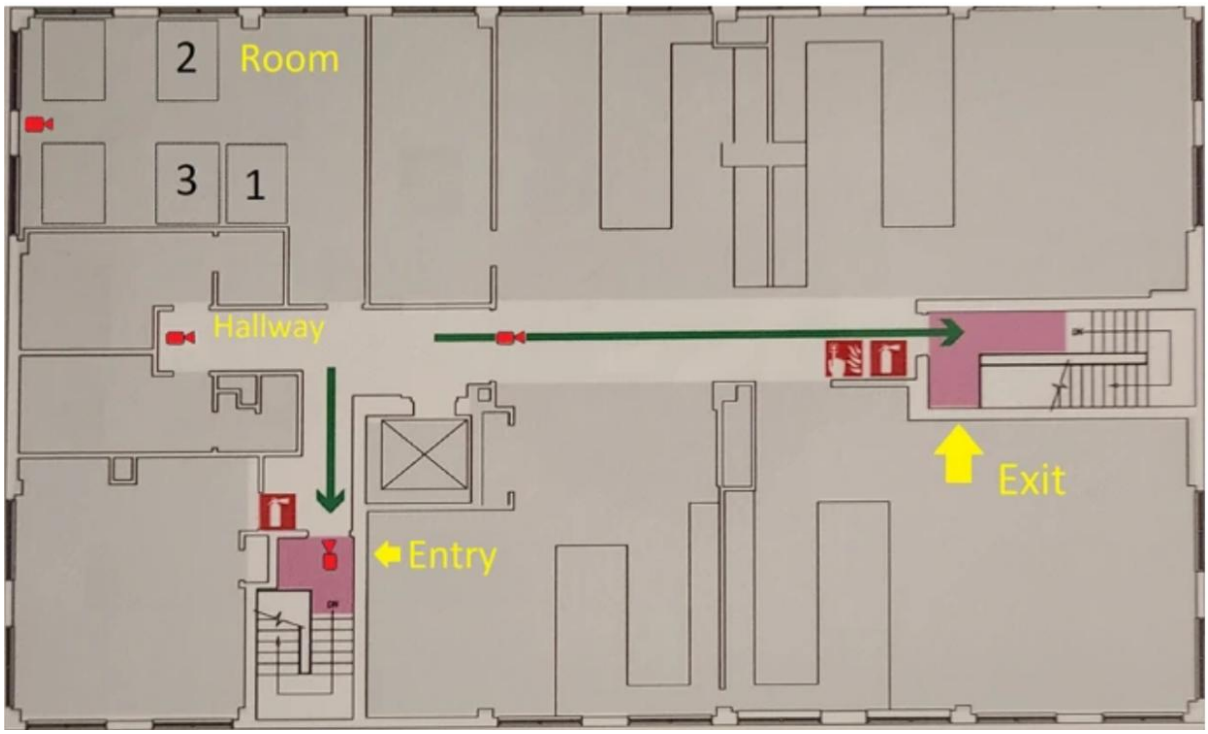
OUTPUT:

```
$ python f1.py
Uniform Cost Search Path:
Syracuse -> New York -> Philadelphia -> Pittsburgh -> Columbus -> Indianapolis -> Chicago
Total cost: 1199
Number of paths explored: 10

Lookup Table (Reached States):
City Index   | City Name       | Cost   | Parent
-----------------------------------------------------
0            | Chicago         | 1199   | Indianapolis
1            | Detroit         | None   | -
2            | Cleveland       | 790    | Pittsburgh
3            | Indianapolis    | 1017   | Columbus
4            | Columbus        | 841    | Pittsburgh
5            | Pittsburgh      | 656    | Philadelphia
6            | Buffalo         | 150    | Syracuse
7            | Syracuse        | 0      | None
8            | New York        | 254    | Syracuse
9            | Philadelphia    | 351    | New York
10           | Baltimore       | 452    | Philadelphia
11           | Boston          | 312    | Syracuse
12           | Providence      | 362    | Boston
13           | Portland        | 419    | Boston
```

Q2 Consider a building floor plan as given in figure. Design a robot guided evacuation planning using the best first search algorithm. Consider the following constraints:

- Allowed movements up, down, left and right
- Walls and rooms block movement
- Evacuation planning covers the route from entry to exit
- Design an evaluation cost function with proper justification for your algorithm




```
#NON HEU. BEFS

def best_first_no_heuristic(grid, start, goal):

    rows = len(grid)
    cols = len(grid[0])

# [cost,node,path]
    open_list = []
    reached = []

    open_list.append([0, start, [start]])
    reached.append(start)

    explored = 0

    while len(open_list) > 0:

        # choose lowest cost node
        open_list.sort()
        cost, current, path = open_list.pop(0)
        explored += 1

        if current == goal:
```

```
        return path, explored

    r, c = current

    moves = [[0,1],[1,0],[0,-1],[-1,0]]

    for m in moves:
        nr = r + m[0]
        nc = c + m[1]

        if nr >= 0 and nr < rows and nc >= 0 and nc < cols:
            if grid[nr][nc] == 0:

                nxt = (nr, nc)

                if nxt not in reached:
                    reached.append(nxt)
                    open_list.append(
                        [cost + 1, nxt, path + [nxt]]
                    )

  return None, explored


#making the map
building = [
[1,1,1,1,1,1,1,1,1,1],
[1,0,0,0,1,0,0,0,0,1],
[1,0,1,0,1,0,1,1,0,1],
[1,0,1,0,0,0,0,1,0,1],
[1,0,1,1,1,1,0,1,0,1],
[1,0,0,0,0,0,0,0,0,1],
[1,1,1,1,1,1,1,1,1,1]
]
#1 is for the wall
#0 if for path

start = (1,1)
exit = (5,8)

path, explored = best_first_no_heuristic(building, start, exit)
```

```
print("Evacuation Path:", path)
print("Nodes Explored:", explored)
```

OUTPUT:-

```
$ python Q2.py
Evacuation Path: [(1, 1), (1, 2), (1, 3), (2, 3), (3, 3), (3, 4), (3, 5), (3, 6), (4, 6), (5, 6), (5, 7), (5, 8)]
Nodes Explored: 25
```